```tsx
import React, {
  forwardRef,
  useState,
  useRef,
  useCallback,
  useImperativeHandle,
} from "react";
import { UploadCloud, File, Trash2, LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface FileUploadActionButtonConfig {
  icon: LucideIcon;
  onClick: (file: File) => void;
  tooltipText?: string;
  disabled?: boolean;
}

export interface FileUploadProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange"
> {
  label?: string;
  error?: string;
  maxSizeInBytes?: number;
  acceptTypes?: string[];
  onChange?: (files: File[]) => void;
  actionButtons?: FileUploadActionButtonConfig[];
}

export interface FileUploadRef {
  clearFiles: () => void;
  getFiles: () => File[];
}

export interface UploadedFile {
  file: File;
  id: string;
  preview?: string;
}

export const FileUpload = forwardRef<FileUploadRef, FileUploadProps>(
  (
    {
      label,
      error,
```

```
      maxSizeInBytes = 10 * 1024 * 1024,
      acceptTypes = [],
      onChange,
      actionButtons = [],
      className,
      disabled,
      required,
      ...props
    },
    ref,
  ) => {
    const fileInputRef = useRef<HTMLInputElement>(null);
    const dropZoneRef = useRef<HTMLDivElement>(null);
    const hasError = Boolean(error);
    const [files, setFiles] = useState<UploadedFile[]>([]);
    const [isDragOver, setIsDragOver] = useState(false);
    const [isFocused, setIsFocused] = useState(false);

    useImperativeHandle(ref, () => ({
      clearFiles: () => {
        setFiles([]);
        if (fileInputRef.current) {
          fileInputRef.current.value = "";
        }
      },
      getFiles: () => files.map((f) => f.file),
    }));

    const formatFileSize = useCallback((bytes: number): string => {
      if (bytes === 0) return "0 Bytes";
      const k = 1024;
      const sizes = ["Bytes", "KB", "MB", "GB", "TB"];
      const i = Math.floor(Math.log(bytes) / Math.log(k));
      return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + " " + sizes[i];
    }, []);

    const validateFile = useCallback(
      (file: File): string | null => {
        if (maxSizeInBytes && file.size > maxSizeInBytes) {
          return `File size exceeds maximum allowed size of
${formatFileSize(maxSizeInBytes)}`;
        }
        if (
          acceptTypes.length > 0 &&
          !acceptTypes.some((type) => file.type.match(type.replace("*", ".*")))
        ) {
          return `File type ${file.type} is not allowed`;
        }
        return null;
      },
```

```
      [maxSizeInBytes, acceptTypes, formatFileSize],
    );

    const processFiles = useCallback(
      (fileList: FileList | File[]) => {
        const newFiles: UploadedFile[] = [];
        const validFiles: File[] = [];

        Array.from(fileList).forEach((file) => {
          const validationError = validateFile(file);
          if (validationError) {
            return;
          }

          const id =
`${file.name}-${file.size}-${Date.now()}-${Math.random().toString(36).substr(2,
9)}`;
          let preview: string | undefined;

          if (file.type.startsWith("image/")) {
            preview = URL.createObjectURL(file);
          }

          newFiles.push({ file, id, preview });
          validFiles.push(file);
        });

        if (newFiles.length > 0) {
          setFiles((prev) => [...prev, ...newFiles]);
          onChange?.(validFiles);
        }
      },
      [validateFile, onChange],
    );

    const handleDragOver = useCallback(
      (e: React.DragEvent<HTMLDivElement>) => {
        e.preventDefault();
        e.stopPropagation();
        if (!disabled) {
          setIsDragOver(true);
        }
      },
      [disabled],
    );

    const handleDragLeave = useCallback(
      (e: React.DragEvent<HTMLDivElement>) => {
        e.preventDefault();
        e.stopPropagation();
```

```
    if (!e.currentTarget.contains(e.relatedTarget as Node)) {
      setIsDragOver(false);
    }
  },
  [],
);

const handleDrop = useCallback(
  (e: React.DragEvent<HTMLDivElement>) => {
    e.preventDefault();
    e.stopPropagation();
    setIsDragOver(false);

    if (disabled) return;

    if (e.dataTransfer.files.length > 0) {
      processFiles(e.dataTransfer.files);
    }
  },
  [disabled, processFiles],
);

const handleFileSelect = useCallback(
  (e: React.ChangeEvent<HTMLInputElement>) => {
    if (e.target.files && e.target.files.length > 0) {
      processFiles(e.target.files);
      e.target.value = "";
    }
  },
  [processFiles],
);

const handleFocus = useCallback(() => {
  setIsFocused(true);
}, []);

const handleBlur = useCallback(() => {
  setIsFocused(false);
}, []);

const removeFile = useCallback((fileId: string) => {
  setFiles((prev) => {
    const fileToRemove = prev.find((f) => f.id === fileId);
    if (fileToRemove?.preview) {
      URL.revokeObjectURL(fileToRemove.preview);
    }
    return prev.filter((f) => f.id !== fileId);
  });
}, []);
```

```
    const limitedActionButtons = actionButtons.slice(0, 4);

    const openFileDialog = useCallback(() => {
      if (!disabled && fileInputRef.current) {
        fileInputRef.current.click();
      }
    }, [disabled]);

    return (
      <div className={cn("w-full", className)}>
        {label && (
          <label
            className={cn(
              "block text-sm font-medium text-text-main mb-1.5",
              disabled && "opacity-50",
            )}
          >
            {label}
            {required && (
              <span className="text-destructive ml-0.5" aria-hidden="true">
                *
              </span>
            )}
          </label>
        )}
        <div
          ref={dropZoneRef}
          className={cn(
            "relative",
            "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
            "border-2 border-dashed
border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
            "rounded-surface",
            "transition-all duration-200 ease-out",
            "focus-within:ring-2 focus-within:ring-amber-500/20
focus-within:border-amber-500 focus-within:bg-amber-500/5",
            "disabled:opacity-50 disabled:pointer-events-none
disabled:cursor-not-allowed",
            hasError &&
              "border-destructive/50 focus-within:ring-destructive/50
focus-within:border-destructive/50",
            disabled && "bg-muted/50",
            isDragOver && "border-amber-500 bg-amber-500/5",
            isFocused && "ring-2 ring-amber-500/20 border-amber-500",
          )}
          onDragOver={handleDragOver}
          onDragLeave={handleDragLeave}
          onDrop={handleDrop}
          onClick={openFileDialog}
          onFocus={handleFocus}
```

```
  onBlur={handleBlur}
  tabIndex={disabled ? -1 : 0}
  role="button"
  aria-label={label || "File upload drop zone"}
  aria-disabled={disabled}
>
  <input
    ref={fileInputRef}
    type="file"
    className="absolute inset-0 w-full h-full opacity-0 cursor-pointer"
    disabled={disabled}
    required={required}
    accept={acceptTypes.join(",")}
    onChange={handleFileSelect}
    multiple
    {...props}
  />
  <div
    className={cn(
      "flex flex-col items-center justify-center p-8",
      "text-center",
      files.length > 0 && "py-4",
    )}
  >
    {files.length === 0 && (
      <>
        <UploadCloud
          className="w-10 h-10 text-muted-foreground/50 mb-3"
          aria-hidden="true"
        />
        <p className="text-text-main font-medium mb-1">
          Drag & drop files here, or click to browse
        </p>
        <p className="text-sm text-muted-foreground/60">
          {acceptTypes.length > 0
            ? `Accepted: ${acceptTypes.join(", ")}`
            : "All file types accepted"}
          {maxSizeInBytes &&
            ` • Max size: ${formatFileSize(maxSizeInBytes)}`}
        </p>
      </>
    )}
    {files.length > 0 && (
      <div className="w-full max-w-md space-y-2">
        {files.map((fileData) => (
          <div
            key={fileData.id}
            className={cn(
              "flex items-center justify-between p-3",
              "bg-background/50",
```

```
                    "rounded-md",
                    "border
border-[color-mix(in_oklch,var(--color-border)_40%,transparent)]",
                    "transition-all duration-150",
                  )}
                >
                  <div className="flex items-center gap-3 min-w-0 flex-1">
                    {fileData.preview ? (
                      <img
                        src={fileData.preview}
                        alt={fileData.file.name}
                        className="w-10 h-10 rounded-md object-cover"
                      />
                    ) : (
                      <File
                        className="w-10 h-10 text-muted-foreground/60
flex-shrink-0"
                        aria-hidden="true"
                      />
                    )}
                    <div className="min-w-0">
                      <p className="text-sm font-medium text-text-main truncate">
                        {fileData.file.name}
                      </p>
                      <p className="text-xs text-muted-foreground/60">
                        {fileData.file.type || "Unknown type"} •{" "}
                        {formatFileSize(fileData.file.size)}
                      </p>
                    </div>
                  </div>
                  <div className="flex items-center gap-1.5 shrink-0">
                    {limitedActionButtons.length > 0 &&
                      limitedActionButtons.map((action, index) => (
                        <button
                          key={index}
                          type="button"
                          className={cn(
                            "relative flex items-center justify-center",
                            "w-8 h-8 rounded-md",
                            "text-muted-foreground/60 hover:text-foreground",
                            "bg-transparent hover:bg-accent",
                            "transition-transform duration-100",
                            "active:scale-95",
                            "focus:outline-none focus-visible:ring-2
focus-visible:ring-amber-500/20 focus-visible:border-amber-500",
                            "disabled:opacity-50 disabled:pointer-events-none",
                          )}
                          aria-label={action.tooltipText}
                          disabled={disabled || action.disabled}
                          onClick={(e) => {
```

```
                            e.stopPropagation();
                            action.onClick(fileData.file);
                          }}
                        >
                          <action.icon
                            className="w-4 h-4"
                            aria-hidden="true"
                          />
                        </button>
                      ))}
                    <button
                      type="button"
                      className={cn(
                        "relative flex items-center justify-center",
                        "w-8 h-8 rounded-md",
                        "text-muted-foreground/60 hover:text-destructive",
                        "bg-transparent hover:bg-destructive/10",
                        "transition-transform duration-100",
                        "active:scale-95",
                        "focus:outline-none focus-visible:ring-2
focus-visible:ring-destructive focus-visible:ring-offset-2",
                        "disabled:opacity-50 disabled:pointer-events-none",
                      )}
                      aria-label="Remove file"
                      disabled={disabled}
                      onClick={(e) => {
                        e.stopPropagation();
                        removeFile(fileData.id);
                      }}
                    >
                      <Trash2 className="w-4 h-4" aria-hidden="true" />
                    </button>
                  </div>
                </div>
              ))}
            </div>
          )}
        </div>
      </div>
      {error && (
        <p
          className={cn(
            "mt-1.5 text-sm",
            "text-destructive/90",
            "font-medium",
          )}
          role="alert"
        >
          {error}
        </p>
```

```
        )}
      </div>
    );
  },
);

FileUpload.displayName = "FileUpload";
```